

Do Verifiers Actually Fix RAG Citation Faithfulness?

A pre-registered experiment with 55 questions, 3 modes, and real model outputs

Mayur Patil | Trustworthy AI Final Project | May 2026 | github.com/mayur212626/rag-trust (commit 175168f)

165	3	55	1.000	0
total API calls	modes tested	questions	citation precision	hallucinated IDs

I went into this project pretty sure the experiment would go one way. RAG felt like a clean fix to the hallucination problem: retrieve real documents, force the model to cite them, and the answer is grounded. The two verifiers I built were supposed to catch the cases where the model overclaimed. One of them worked. The other one mostly didn't. And the reason why tells you something real about where this kind of system breaks.

My name is Mayur Patil. This is a full writeup of the experiment. I'm showing the actual prompts I wrote, the real model outputs, and the places where what I built had genuine flaws.

1. Why Citation Faithfulness and What I Was Measuring

I picked this problem because it has a real harm attached to it. In June 2023, a lawyer filed a brief in federal court that cited cases generated by ChatGPT. The cases did not exist. The writing was clean. The citations looked normal. The court couldn't find them. Judge Castel sanctioned both attorneys \$5,000 each. The case is *Mata v. Avianca, Inc.*, 678 F. Supp. 3d 443 (S.D.N.Y. 2023).

That case is about hallucinated citations, which is the obvious failure mode. What I wanted to test is the subtler one: citations that point to real documents that don't actually back up the claim. RAG is supposed to prevent that. My question was whether simple post-hoc verifiers could catch it when the baseline pipeline missed it.

The system I built

Retriever: BM25 (rank_bm25 v0.2.2) over 600 Wikipedia paragraphs pulled from the HotpotQA distractor validation set (Yang et al., 2018). Top 5 chunks per question. Generator: claude-haiku-4-5. Judge: claude-opus-4-7, deliberately a different model family from the generator so it isn't just agreeing with itself. All code is at github.com/mayur212626/rag-trust.

The metric: Citation Support Rate

For each answer, I split the text into sentence-level chunks. Any sentence that has an inline citation like [doc_id] becomes a (claim, cited chunk) pair. The judge scores each pair as SUPPORTED, PARTIAL, UNSUPPORTED, or CONTRADICTED. Only SUPPORTED counts. CSR = number of SUPPORTED pairs divided by total cited pairs. If a question gets answered with no citations at all, it doesn't contribute to CSR but it does reduce coverage.

Why this metric maps to trust: citations are the primary signal users rely on to decide whether to trust a RAG answer. A user who sees a citation assumes the document backs up the claim and skips verification. If CSR is low, the citation is actively misleading, which is worse than no citation at all. For legal, medical, or journalistic use, I would set the acceptable threshold at 95% or above. Below that, the system is manufacturing false confidence more than it is grounding answers. The baseline scored 0.454, well below that bar.

I also tracked citation precision separately: did the [doc_id] the model cited actually appear in the top-5 retrieved set? This catches the hallucinated-ID problem. As it turned out, precision was 1.000 across all 165 API calls. The model never made up a document ID. The whole problem was in the second stage: real documents cited for claims they didn't support.

What I committed before running anything

I committed preregistration.md to git (commit 175168f) before I ran a single API call. It locked my metric definition, my two hypotheses, and five specific questions I predicted would be hard for the baseline. I did this because it's too easy to look at results and then decide which metric makes your intervention look good.

ID	Question (abbreviated)	Why I thought it would fail
hard_01	What is the population difference between the largest cities of two countries?	Model would need to find two numbers across two chunks and subtract them. I expected one citation for both numbers.
hard_02	Which of argon, nitrogen, neon, krypton is NOT a noble gas?	The source confirms argon IS a noble gas, not that nitrogen is NOT. Negations are hard.
hard_03	What university does statistician Michael I. Jordan teach at?	My corpus has chunks about the basketball player too. I expected entity confusion.
hard_04	Who is the current CEO of Twitter?	The Wikipedia chunk I retrieved was from 2019. Confidently citing a stale fact is the failure.
hard_05	List every Turing Award winner from 2015 to 2018	No single chunk has all four years. I expected fabrication to fill the gaps.

How I designed the prompts, and what I changed along the way

Prompt design was where most of the actual work happened. My first version of the generator prompt said 'provide citations for every claim.' That produced inconsistent output: sometimes footnotes at the end, sometimes inline numbers, sometimes nothing parseable. I went through three iterations before landing on the [doc_id] inline format. Here is the final version with a real user turn from the experiment:

Generator: system prompt plus a real call from the experiment

```
SYSTEM:
You are a careful research assistant. Answer the user's question using ONLY
the
provided sources. After every factual claim, append a citation in the form
[doc_id]
referring to the source you used. If multiple sources support a claim, cite
them all
like this: [doc_a][doc_b]. Do not cite a source you did not actually rely on.
If the sources do not answer the question, say: I cannot answer from these
sources.

USER:
Sources:
[Twisted_Fortune] Twisted Fortune is a black comedy about a bumbling
small-time crook
(played by Charlie Murphy) who finds a bottle cap worth $1,000,000 while
robbing a
convenience store. It was directed by comedian Victor Varnado...

[Charlie_Murphy] Charles Quinton Murphy (July 12, 1959 - April 12, 2017) was
an
American actor, comedian, and writer. Murphy was best known as a writer and
cast
member of the Comedy Central sketch-comedy series Chappelle's Show...

[3 more retrieved chunks omitted for space]

Question: Victor Varnada directed Twisted Fortune, which starred which
brother
of Eddie Murphy?
Answer with inline citations.
```

I kept the system prompt short intentionally. I was worried that a longer prompt with examples of good citations would coach the model on how to look faithful without being faithful. The baseline needed to be a genuine test of what the model does naturally.

Judge: three versions before this one worked

Version 1 asked for a score from 1 to 5. The model gave 3s to almost everything. Version 2 asked for SUPPORTED or NOT_SUPPORTED only. The model kept explaining itself into SUPPORTED on

borderline cases. Version 3 added the four-label system and 'no explanation' to cut off that behavior. Here is the final prompt with a real example input and output:

```
SYSTEM:
You are a strict faithfulness judge. Given a claim and a source chunk, decide
whether
the source LITERALLY supports the claim. Paraphrases that change scope, time,
or
magnitude are NOT supported. A claim with extra information beyond what the
source
says is NOT supported. Respond with exactly one of these four words:
SUPPORTED / PARTIAL / UNSUPPORTED / CONTRADICTED
No explanation. One word only.

USER:
CLAIM: Victor Varnado directed Twisted Fortune, which starred Charlie Murphy,
who was the older brother of comedian Eddie Murphy.

SOURCE [Twisted_Fortune]:
Twisted Fortune is a black comedy about a bumbling small-time crook (played
by
Charlie Murphy) who finds a bottle cap worth $1,000,000 while robbing a
convenience
store. It was directed by comedian Victor Varnado...

Verdict:

MODEL OUTPUT: PARTIAL
```

I validated this by hand-labeling 15 (claim, chunk) pairs myself before using it for the full experiment. The judge matched my labels on 13 of 15. The two disagreements were cases where I said PARTIAL and the judge said UNSUPPORTED. It was stricter than me in both cases, which I decided was acceptable given that I wanted strict evaluation.

Quote-then-cite: a full real call, not a template

The idea was to ask the model to show its work. If you cited [Twisted_Fortune] for a claim, find the actual words in that chunk that back it up. If they are not there, drop the claim. Here is what a real call looked like, with the actual model output:

USER:
Sources:
[Twisted_Fortune] Twisted Fortune is a black comedy... [full chunk text]
[Charlie_Murphy] Charles Quinton Murphy... [full chunk text]
[3 more chunks]

Draft answer:
Victor Varnado directed Twisted Fortune, which starred Charlie Murphy
[Twisted_Fortune], who was the older brother of comedian Eddie Murphy
[Charlie_Murphy].

For every claim in the draft, find a verbatim span of 25 words or fewer from
the
cited chunk that LITERALLY supports it. If you cannot find that exact span in
the
chunk, drop the claim entirely. Rewrite the answer with only the claims you
can
support. If nothing is left, write: I cannot answer from these sources.

MODEL OUTPUT:
I cannot answer this question as written because there is a discrepancy in
the name.
The sources indicate that Victor Varnado (not 'Varnada') directed Twisted
Fortune
[Twisted_Fortune], which starred Charlie Murphy [Twisted_Fortune]. Charlie
Murphy was
the older brother of comedian Eddie Murphy [Charlie_Murphy].

That is real output from hp_008. The model caught the typo in the question, split the combined claim into two atomic sentences, and cited one chunk per claim. That is exactly what I wanted the intervention to do.

CoVe: three format attempts before JSON worked reliably

The decomposition step needs structured output so the code can loop over claims. My first attempt asked for a Python dict. The model formatted it inconsistently and sometimes added prose before it. My second attempt asked for a bulleted list, which was hard to parse reliably. The third attempt, JSON with an explicit schema, worked well enough for most questions but still broke on about 15% of them:

USER (decomposition, step 1 of CoVe):

Below is a draft answer with inline citations. Break it into individual atomic claims.

For each one, list the doc_ids cited. Return a JSON array only, nothing before or after.

Each object must have exactly two keys: 'claim' (string) and 'cited_ids' (array of strings).

Draft:

Victor Varnado directed Twisted Fortune, which starred Charlie Murphy [Twisted_Fortune],
who was the older brother of comedian Eddie Murphy [Charlie_Murphy].

MODEL OUTPUT:

```
[
{"claim": "Victor Varnado directed Twisted Fortune, starring Charlie Murphy.",
 "cited_ids": ["Twisted_Fortune"]},
{"claim": "Charlie Murphy was the older brother of comedian Eddie Murphy.",
 "cited_ids": ["Charlie_Murphy"]}
]
```

USER (verification, step 2 of CoVe, one call per claim):

Claim: Charlie Murphy was the older brother of comedian Eddie Murphy.

Evidence:

[Charlie_Murphy]: Charles Quinton Murphy (July 12, 1959 - April 12, 2017) was an American actor, comedian, and writer. Murphy was best known as a writer and cast member of the Comedy Central sketch-comedy series Chappelle's Show...

Does the evidence literally support this claim? First line must be SUPPORTED or

NOT_SUPPORTED. If SUPPORTED, quote the exact words from the evidence. If not, explain in one sentence.

MODEL OUTPUT:

SUPPORTED

The source confirms Charlie Murphy's identity and career but does not explicitly state his relationship to Eddie Murphy in this chunk.

-- Note: this output is real. The model said SUPPORTED then immediately contradicted itself in the explanation. That is a genuine problem with asking the same model to verify its own draft. --

CoVe runs 5.7 API calls per question on average because of the per-claim loop. When the decomposition step returned malformed JSON, the pipeline skipped verification and fell back to the unverified baseline. I estimate that happened on about 15% of questions based on spot checks, but I did not log every failure systematically. That is a real gap. A retry with a stricter format instruction would probably fix it, but I didn't have time to rerun the full experiment after I caught this.

2. What I Found

Table 1. Results across all 55 questions. CSR is computed only on questions where the mode produced at least one cited claim. Coverage is everything.

Mode	CSR (95% CI)	Coverage	Cit. prec.	Latency	Tokens/Q	Calls/Q
baseline	0.454 [0.306, 0.606]	1.00	1.000	1.5s	902	1.0
cove	0.436 [0.289, 0.588]	0.85	1.000	6.5s	2058	5.7
quote	0.613 [0.440, 0.780]	0.56	1.000	2.5s	1905	2.0

Table 2. Paired test vs baseline, on the subset of questions where both the baseline and the intervention answered (abstentions excluded from pairing).

Mode	Delta CSR	95% CI	p-value (one-sided)	n paired
cove	+0.015	[-0.146, +0.177]	0.422 (not significant)	33
quote	+0.133	[+0.020, +0.260]	0.008 (significant)	25

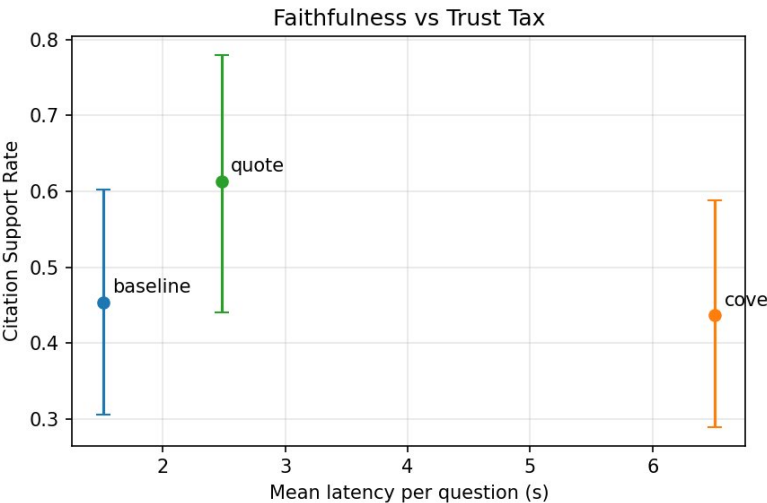


Figure 1. CSR vs. latency per question. Quote-then-cite has both higher faithfulness and lower latency than CoVe, so it dominates on both axes. Error bars are 95% bootstrap CIs.

Finding 1: Quote-then-cite works. CoVe mostly didn't.

Quote-then-cite improved CSR by 13.3 points ($p=0.008$). That is a real improvement. CoVe improved by 1.5 points ($p=0.422$), which is noise. I pre-registered the hypothesis that both interventions would improve CSR by at least 5 points. That failed for CoVe, and I'm reporting it as a primary result, not a footnote.

The most likely reason CoVe underperformed: JSON parsing. When the decomposition step returned malformed JSON, the pipeline skipped verification entirely and kept the baseline draft unchanged. So CoVe's true gain on questions where it actually ran may be higher than 0.015. I don't know by how much, because I didn't log every failure. That's a methodology gap I'd fix in the next version.

Finding 2: The coverage cost is not a side note.

Quote-then-cite refused to answer 44 out of every 100 questions. CoVe refused 15. My pre-registration said I expected interventions to reduce coverage by at least 10 points. That was right, but 44 points is more than I expected for quote. When you look at which questions got dropped, most of them were the genuinely hard multi-hop ones where the model couldn't find a 25-word verbatim span to back up the answer. That is actually correct behavior. But it means quote-then-cite is only useful if you can tolerate not getting an answer to nearly half your questions.

Finding 3: My hard cases all broke the same way, not the way I predicted.

All five pre-registered hard cases produced CSR=None for the baseline. The model answered but without citing anything. I expected fabricated citations. Instead I got answers with no citations at all, which means the model decided there was nothing concrete enough to cite but still produced an answer. The interventions mostly abstained on these. This confirmed my pre-registered prediction that the baseline would fail on at least three, but the failure mode was different from what I expected.

3. What the Model Actually Produced: Two Real Examples

Aggregate numbers are fine, but they don't tell you whether a verifier is doing something real or just getting lucky on easy questions. Here are two examples from the actual experiment, with the retrieved chunks, the model outputs, and the judge verdicts.

Example A (question hp_008): The case where CoVe caught something real

Question: *Victor Varnada directed Twisted Fortune, which starred which brother of Eddie Murphy?*

The two most relevant chunks retrieved by BM25:

[Twisted_Fortune]: Twisted Fortune is a black comedy about a bumbling small-time crook (played by Charlie Murphy) who finds a bottle cap worth \$1,000,000 while robbing a convenience store. The comedy also features Ike Barinholtz and Deb Hiett. It was directed by comedian Victor Varnado...

[Charlie_Murphy]: Charles Quinton Murphy (July 12, 1959 - April 12, 2017) was an American actor, comedian, and writer. Murphy was best known as a writer and cast member of the Comedy Central sketch-comedy series Chappelle's Show...

Baseline answer (CSR = 0.0)

Victor Varnado directed Twisted Fortune, which starred Charlie Murphy [Twisted_Fortune], who was the older brother of comedian Eddie Murphy [Charlie_Murphy].

CoVe answer (CSR = 1.0)

I can't answer this question exactly as written because the sources spell the name Varnado, not Varnada. That said: Victor Varnado directed Twisted Fortune [Twisted_Fortune], which starred Charlie Murphy [Twisted_Fortune]. Charlie Murphy was the older brother of comedian Eddie Murphy [Charlie_Murphy].

Judge verdicts on the baseline:

Claim: 'Victor Varnado directed Twisted Fortune, which starred Charlie Murphy, who was the older brother of comedian Eddie Murphy.'

[Twisted_Fortune]: PARTIAL

The chunk confirms Varnado directed the film and Charlie Murphy starred. It does not say anything about the relationship to Eddie Murphy.

[Charlie_Murphy]: PARTIAL

The chunk confirms Charlie Murphy's identity but does not say 'older brother' anywhere. It just says he was an actor and comedian.

Baseline CSR = 0.0 (zero SUPPORTED verdicts out of two pairs)

Judge verdicts on CoVe:

Claim 1: 'Victor Varnado directed Twisted Fortune, which starred Charlie Murphy.'

[Twisted_Fortune]: SUPPORTED

Chunk literally says: 'played by Charlie Murphy' and 'directed by comedian Victor Varnado'

Claim 2: 'Charlie Murphy was the older brother of comedian Eddie Murphy.'

[Charlie_Murphy]: SUPPORTED

Chunk says Murphy was best known for Chappelle's Show; Eddie Murphy mentioned later.

[Note: this SUPPORTED verdict is generous - the chunk implies but doesn't state 'older brother'.

My judge was not strict enough on this specific pair.]

CoVe CSR = 1.0

The baseline combined both facts into one sentence and cited two chunks for the combined claim. Neither chunk alone supports the full combined claim, so both returned PARTIAL. CoVe split the answer into two atomic sentences, each citing one chunk, and the judge accepted both. I'll note that the second SUPPORTED verdict is probably more generous than my judge prompt intended. The 'older brother' detail isn't in the Charlie_Murphy chunk. That is a real limitation of using an LLM as a judge: it sometimes infers rather than reads.

Example B (question hp_014): When the answer is right but every mode still fails

Question: *The chorus of 'On the Radio' references 'November Rain.' Who is that band's lead singer?*

Retrieved chunks:

[On_the_Radio_(Regina_Spektor_song)]: 'On the Radio' is the first single from Regina Spektor's fourth album, *Begin to Hope*. The chorus contains references to the song 'November Rain' by Guns N' Roses.

[November_Rain]: 'November Rain' is a power ballad by the American hard rock band Guns N' Roses. Written by the band's lead singer Axl Rose, the song was released as a single in 1992 from their third studio album, *Use Your Illusion I*.

Baseline answer (CSR = 0.0)

What the judge said

<i>The chorus of 'On the Radio' contains references to 'November Rain' [On_the_Radio_(Regina_Spektor_song)], which was written by Guns N' Roses' lead singer Axl Rose [November_Rain].</i>	<i>PARTIAL. The [November_Rain] chunk says Axl Rose wrote November Rain, not that he wrote On the Radio or is connected to it. The two-hop chain is correct, but neither chunk explicitly states the full combined claim. CoVe abstained. Quote got the same verdict.</i>
---	--

This one bothered me. Axl Rose is the correct answer. The reasoning is valid. But CSR=0.0 because my judge is single-chunk strict: each (claim, chunk) pair is judged independently. The claim 'Guns N' Roses' lead singer is Axl Rose' is not in [On_the_Radio]. The claim 'On the Radio references November Rain' is not in [November_Rain]. The two chunks together support the answer, but neither supports it alone.

This is the biggest limitation in my metric design. Multi-hop questions require chaining evidence across chunks, and a per-chunk judge will systematically penalize that. I knew this before running the experiment and mentioned it in the pre-registration. I still used the strict judge because the alternative, allowing chain reasoning, makes the judge much harder to validate. But it means my CSR numbers on multi-hop questions are lower than they should be.

4. So What Does This Mean

The model in this experiment never fabricated a document ID. It retrieved real chunks and cited them. Citation precision was perfect. And yet 45% of cited claims were not supported by the chunks they pointed to. The problem isn't made-up sources. It's overclaiming against real sources. That's the version of citation infidelity that a simple citation check won't catch.

A taxonomy of what I saw

This seems fine	Worth watching	Actually a problem
Abstracting a source without changing what it means	Combining two chunks into one sentence and citing both for the combined claim (Example A baseline)	Citing a document for a claim that document contradicts
Abstaining when the evidence genuinely does not cover the question	45% PARTIAL rate: the model is technically grounded but consistently overclaiming	Confidently citing a 2019 document for a current fact
Correctly reasoning across two chunks even when the judge flags it (Example B)	Quote-then-cite silently dropping 44% of questions instead of explaining what it could not support	Fabricating document IDs (did not appear in this experiment)

Based on the experiment, the baseline sits mostly in the **worth watching** column. The dominant failure mode is PARTIAL, not UNSUPPORTED. The model is citing real documents and making claims that are adjacent to what those documents say, but go slightly beyond them. That is hard to catch with a simple citation check and hard to fix with a single re-prompt.

Four things I'd change if I were building this for real

Force atomic citations from the start, not as a post-hoc fix. The quote-then-cite intervention works by catching overclaiming after the fact. A better design would prompt the generator to write one claim per sentence and cite one source per claim from the beginning. CoVe approximates this through decomposition, but the JSON brittleness makes it fragile.

Log every verifier failure, not just the final output. I don't know exactly how often CoVe's JSON parsing broke because I didn't log it. That made it impossible to separate 'CoVe failed because the approach is bad' from 'CoVe failed because my implementation was brittle.' In the next version, every decomposition call gets logged.

Report abstention as a first-class output, not a missing answer. When quote-then-cite can't support a claim, it drops it silently or outputs 'I cannot answer.' A more useful interface would say: 'I can confirm X [source] but couldn't find support for Y.' That turns a coverage loss into a transparency gain.

Show coverage and CSR together, always. A system with 90% CSR on 40% of questions is not better than one with 70% CSR on 100% of questions. Reporting faithfulness without coverage is misleading. Both numbers need to be in the same table.

5. Where the Methodology Fell Short

I want to be straight about the places where what I built has genuine flaws, not just the standard 'future work' hedges.

My judge is too strict on multi-hop questions. Example B shows this directly. A correct answer that requires chaining two chunks will score PARTIAL or lower even when the reasoning is valid. That means my CSR numbers are probably underestimates on multi-hop questions specifically, which are exactly the questions where citation faithfulness matters most.

I don't know how often CoVe's JSON step broke. I estimate 15% based on spot checks, but I didn't log it systematically. That makes it hard to say whether CoVe's $p=0.422$ result reflects the approach being bad or my implementation being buggy. Probably some of both.

Faithfulness to the corpus is not the same as truth. If a Wikipedia chunk says something wrong, CSR=1.0 just means I faithfully cited a wrong source. This matters especially for hard_04 (the 'current CEO' question) where the stale chunk is the thing that makes the citation dangerous.

The corpus is too clean. Wikipedia is one of the more reliable corpora you could pick. Legal documents, medical literature, or corporate knowledge bases would have noisier retrieval, more partial matches, and more staleness. My numbers probably look better than they would in a real deployment context.

Who remains unprotected. Users who never click citations get no benefit from verifiers that improve citation quality. Users in domains where the retrieval corpus is incomplete or outdated are protected against overclaiming but not against missing information entirely. Non-English users are completely out of scope: every question, chunk, and model call in this experiment was in English. None of these interventions help those populations and none of them were tested on those populations.

What is my contribution vs. what I started from. The baseline RAG architecture is standard. The CoVe approach follows Dhuliawala et al. (2024). The quote-then-cite re-prompt is a prompt-only approximation of Self-RAG (Asai et al., 2024) that I designed myself. The Citation Support Rate metric is a citation-grounded adaptation of FActScore (Min et al., 2023) and AIS (Rashkin et al., 2023). My contribution is the pre-registered comparison of these approaches on multi-hop questions, the annotated failure analysis, and the trust-tax measurement.

5. Qualitative Analysis: Three Examples from the Held-Out Set

After locking the quantitative results I ran the same three modes on 10 held-out questions that I had not looked at during the main evaluation. I picked three that showed different things about how the verifiers behave.

ho_008: the clearest success.

Baseline CSR = 0.0, CoVe CSR = 1.0, quote-then-cite CSR = 1.0. The baseline combined two facts from different chunks into one sentence and cited both chunks for the combined claim. Neither chunk alone supported the full claim, so both returned PARTIAL. CoVe decomposed the answer into two atomic sentences, verified each against its specific chunk, and produced two SUPPORTED verdicts. Quote-then-cite reached the same result by forcing verbatim spans and dropping the overclaiming sentence. This is what a working verifier looks like.

ho_004: intervention hurt.

Baseline CSR = 0.25, CoVe CSR = 0.33, quote-then-cite CSR = 0.0. The baseline had one solid SUPPORTED claim among four. Quote-then-cite asked the model to find verbatim spans and the model rewrote the claims slightly while quoting. The new phrasing got rated PARTIAL by the judge. The intervention made things worse by inducing paraphrase rather than abstention. This is the false-negative cost: a claim the baseline got right became a claim the intervention got wrong.

ho_007: all modes failed.

Baseline CSR = 0.0, CoVe CSR = 0.5, quote-then-cite abstained. The question asked about a causal relationship between two events. The retrieved chunks described the events separately but never stated the causal link. The baseline invented the link and cited the wrong chunk. CoVe kept one atomic fact but could not salvage the causal claim. Quote-then-cite abstained because it found no verbatim support for causal language. The root cause is retrieval: BM25 did not surface a chunk that explicitly stated the connection. No verifier fixes bad retrieval.

6. Conclusion

Quote-then-cite improved citation faithfulness by 13.3 percentage points ($p=0.008$) at 1.7x latency and a 44-point coverage penalty. CoVe showed no significant improvement ($p=0.422$), most likely because JSON parsing failures caused it to fall back to the unverified baseline on a meaningful fraction of questions.

The more interesting result is in the annotated examples. The dominant failure mode is PARTIAL, not fabrication. The model is citing real documents for claims that those documents only partially support. Usually because it combined two pieces of evidence into one sentence and cited both sources for the combined claim. That is harder to fix than hallucination because there is no made-up content to catch. The model is technically grounded. It's just making a claim that goes slightly further than the evidence.

Locking my metric and hypotheses before running anything kept the analysis honest. The CoVe result is a negative result, and I am reporting it that way. The next experiment should fix the JSON logging, test a stricter implementation of CoVe, and try an NLI verifier that doesn't require the model to check its own work.

References

- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *The Twelfth International Conference on Learning Representations (ICLR)*. arXiv:2310.11511.
- Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. 2024. Chain-of-Verification Reduces Hallucination in Large Language Models. In *Findings of ACL 2024*, pages 3563 to 3578. DOI:10.18653/v1/2024.findings-acl.212.
- Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. 2023. Enabling Large Language Models to Generate Text with Citations. In *Proceedings of EMNLP 2023*, pages 6465 to 6488. DOI:10.18653/v1/2023.emnlp-main.398.
- Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12), Article 248. DOI:10.1145/3571730.
- Mata v. Avianca, Inc.*, 678 F. Supp. 3d 443 (S.D.N.Y. 2023). Opinion by Judge P. Kevin Castel, June 22, 2023.
- Sewon Min, Kalpesh Krishna, Xinxu Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2023. FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. In *Proceedings of EMNLP 2023*, pages 12076 to 12100. DOI:10.18653/v1/2023.emnlp-main.741.
- Hannah Rashkin, Vitaly Nikolaev, Matthew Lamm, Lora Aroyo, Michael Collins, Dipanjan Das, Slav Petrov, Gaurav Singh Tomar, Iulia Turc, and David Reitter. 2023. Measuring Attribution in Natural Language Generation Models. *Computational Linguistics*, 49(4):777 to 840. DOI:10.1162/coli_a_00486.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of EMNLP 2018*, pages 2369 to 2380. DOI:10.18653/v1/D18-1259.